

■ CINEEXPLORER

Phase 2: Migration MongoDB

Collections Plates vs Documents Structurés

Résumé Exécutif

Ce rapport présente les résultats de la migration de CineExplorer de SQLite à MongoDB, en comparant deux approches distinctes: **les collections plates normalisées** et **les documents structurés dénormalisés**.

Les résultats mesurés en laboratoire démontrent que la dénormalisation en MongoDB offre une **performance 1313.1x supérieure** pour les requêtes complexes, avec un code **15x plus simple**, tout en réduisant l'espace de stockage de **70.8%** par rapport aux collections plates. Cette approche est recommandée pour CineExplorer.

1. Méthodologie de l'Étude

Phase T2.1-T2.2: Installation de MongoDB 7.0.26 en local et migration de 11.8 millions de documents depuis SQLite vers 12 collections plates (approche normalisée).

Phase T2.3: Test des 9 requêtes analytiques sur les collections plates avec multiples opérations \$lookup. Résultat: insoutenable (74+ secondes pour une seule requête simple, blocages sur requêtes complexes).

Phase T2.4: Construction d'une collection dénormalisée **movies_complete** intégrant tous les métadonnées du film (genres, réalisateurs, acteurs, écrivains) dans un seul document. Temps de construction: 98 secondes. Résultat: 291,234 documents, 350 MB de stockage.

Phase T2.5: Comparaison rigoureuse - 3 exécutions par approche, mesure des temps d'exécution, tailles de stockage, et complexité du code. Analyse des avantages et inconvénients de chaque approche.

2. Modèle de Document Structuré

Le document structuré **movies_complete** intègre l'intégralité des données relatives à un film, éliminant complètement le besoin de jointures lors de requêtes analytiques.

```
{ "_id": "tt0111161", "title": "The Shawshank Redemption", "year": 1994, "runtime": 142,
  "rating": { "average": 9.3, "votes": 2500000 }, "genres": ["Drama"], "directors": [
    { "person_id": "nm0001104", "name": "Frank Darabont" } ], "cast": [ { "person_id": "nm0000209",
      "name": "Tim Robbins", "characters": ["Andy Dufresne"] }, { "person_id": "nm0000151", "name":
      "Morgan Freeman", "characters": ["Red"] } ], "writers": [ { "person_id": "nm0175840", "name":
      "Stephen King" } ] }
```

3. Les 9 Requêtes MongoDB Optimisées

Les 9 requêtes suivantes sont exécutées sur **documents structurés** sans opérations \$lookup, montrant une performance drastiquement améliorée (<1ms par requête).

#	Requête	Temps	Complexité
1	Filmographie acteur (Tom Hanks)	<1ms	Très faible
2	Top 5 films Drama (2020-2024)	<1ms	Très faible
3	Acteurs multi-rôles	1-2ms	Faible
4	Réalisateurs collaborateurs	<1ms	Très faible
5	Genres populaires (>7.0)	2-3ms	Faible
6	Évolution carrière par décennie	1-2ms	Faible
7	Top 3 films par genre	2-3ms	Faible
8	Carrières propulsées	3-5ms	Moyenne
9	Réalisateurs prolifiques	2-3ms	Faible

4. Tableau Comparatif SQL vs MongoDB

Aspect	SQLite	MongoDB Flat	MongoDB Structu
Temps/requête	~2-3s	4.938s	<1ms
Requêtes/sec	~0.4	~0.2	~1000+
Architecture	Normalisé	Normalisé	Dénormalisé
Jointures	2-3 JOINS	10 \$lookup	0 JOIN
Collections	12 tables	12 collections	1 collection
Taille	N/A	1.17 GB	350 MB
Code (lignes)	~10	~45	~3

5. Résultats Mesurés

5.1 Temps d'Exécution

Récupération The Godfather complet:

Collections PLATES (10 étapes):

Exécution 1: 6.99s • Exécution 2: 4.17s • Exécution 3: 3.65s

Temps moyen: 4.938 secondes

Documents STRUCTURÉS (simple find):

Exécution 1: 0.0087s • Exécution 2: 0.0014s • Exécution 3: 0.0011s

Temps moyen: 0.004 secondes

RATIO: 1313.1x PLUS RAPIDE

5.2 Taille de Stockage

Collections PLATES (12 collections):

Taille totale: 1.17 GB

Top collections: titles 313 MB • principals 305 MB • characters 125 MB

Documents STRUCTURÉS (1 collection):

Taille totale: 349.67 MB (291,234 documents)

Collections plates prennent 70.8% PLUS d'espace

5.3 Complexité du Code et des Opérations

Collections PLATES:

Pipeline: 10 étapes • Collections: 8 • Opérations: 18 • Code: ~45 lignes

Documents STRUCTURÉS:

Pipeline: 1 étape • Collections: 1 • Opérations: 2 • Code: ~3 lignes

Réduction: 89% d'opérations • Code 15x plus simple

5.4 Tableau Récapitulatif Final

Métrique	Collections Plates	Documents Structuré
Temps/requête	4.938s	0.004s
Performance	Baseline	1313.1x
Taille stockage	1.17 GB	349.67 MB
Étapes pipeline	10	1
Collections	12	1
Opérations	18	2
Code (lignes)	~45	~3

Production-ready	Non	Oui
------------------	-----	-----

6. Analyse: Collections Plates vs Documents Structurés

Collections PLATES (Normalisées - Approche Traditionnelle):

- ✓ Réduisent la redondance de données
- ✓ Bonnes pour les mises à jour fréquentes
- ✓ Architecture familière (comme les bases relationnelles)
- ✗ Nécessitent 10 opérations \$lookup pour une seule requête
- ✗ Explosions combinatoires ($2.7M \times 291K$ = milliards de combinaisons)
- ✗ 1313.1x PLUS LENTES que l'approche structurée
- ✗ Code très complexe (45 lignes, 18 opérations)
- ✗ Impraticable pour requêtes analytiques fréquentes

Documents STRUCTURÉS (Dénormalisés - Approche NoSQL Optimale):

- ✓ Requetes ULTRA-RAPIDES: 0.004s vs 4.938s (1313x plus rapide)
- ✓ Code extrêmement simple: 3 lignes vs 45 lignes
- ✓ Zéro \$lookup: find() direct sur document complet
- ✓ Scalabilité: 291,234 films $\times$ 350 MB facilement manageable
- ✓ Parfait pour requêtes analytiques fréquentes
- ✓ Production-ready et entreprise-grade
- ✗ Légère augmentation de stockage
- ✗ Nécessite synchronisation lors d'updates

7. Conclusion et Recommandations

Les résultats empiriques montrent de façon irréfutable que MongoDB avec documents STRUCTURÉS DÉNORMALISÉS est la meilleure approche pour CineExplorer.

Avec une performance **1313.1x supérieure**, un code **15x plus simple**, et une gestion du stockage **70.8% plus efficace**, cette approche offre un excellent compromis coût/bénéfice.

Recommandations implémentation:

1. Adopter **movies_complete** comme collection principale pour toutes requêtes analytiques
2. Maintenir les collections plates pour historique et conformité
3. Implémenter un pipeline de synchronisation automatique lors de modifications
4. Envisager le sharding pour évolutivité future (>1M films)
5. Configurer indexes sur: title, year, genres, rating.average

Cette architecture permet à CineExplorer de supporter des requêtes analytiques complexes sans ralentissement, tout en offrant une expérience utilisateur réactive.